Mansoura University
Faculty of Computers and Information
Department of Information Technology
Second Semester- 2025-2026

Software Reengineering

Grade: 4 SW

BY: Hala Ahmed

Chapter 4
Reengineering

Outline

- General Idea
- Reengineering Concepts
- A General Model for Software Engineering
  - Types of Changes
  - Software Reengineering Strategies
  - Reengineering Variations
- Reengineering Process
  - Reengineering Approaches
  - Source Code Reengineering Reference Model
  - Phase Reengineering Model

General Idea

- **Reengineering** is the **examination**, **analysis,** and **restructuring of an existing software system** to reconstitute it in **a new form**, and the subsequent implementation of the new form.
- **The goal of reengineering is to:**
  - understand the existing software system artifacts, namely, specification, design, implementation, and documentation, and
  - improve the functionality and quality attributes of the system.
- **Software systems** are reengineered by **keeping one or more of the following four general objectives in mind:**
  - Improving maintainability.
  - Migrating to a new technology.
  - Improving quality.
  - Preparing for functional enhancement.

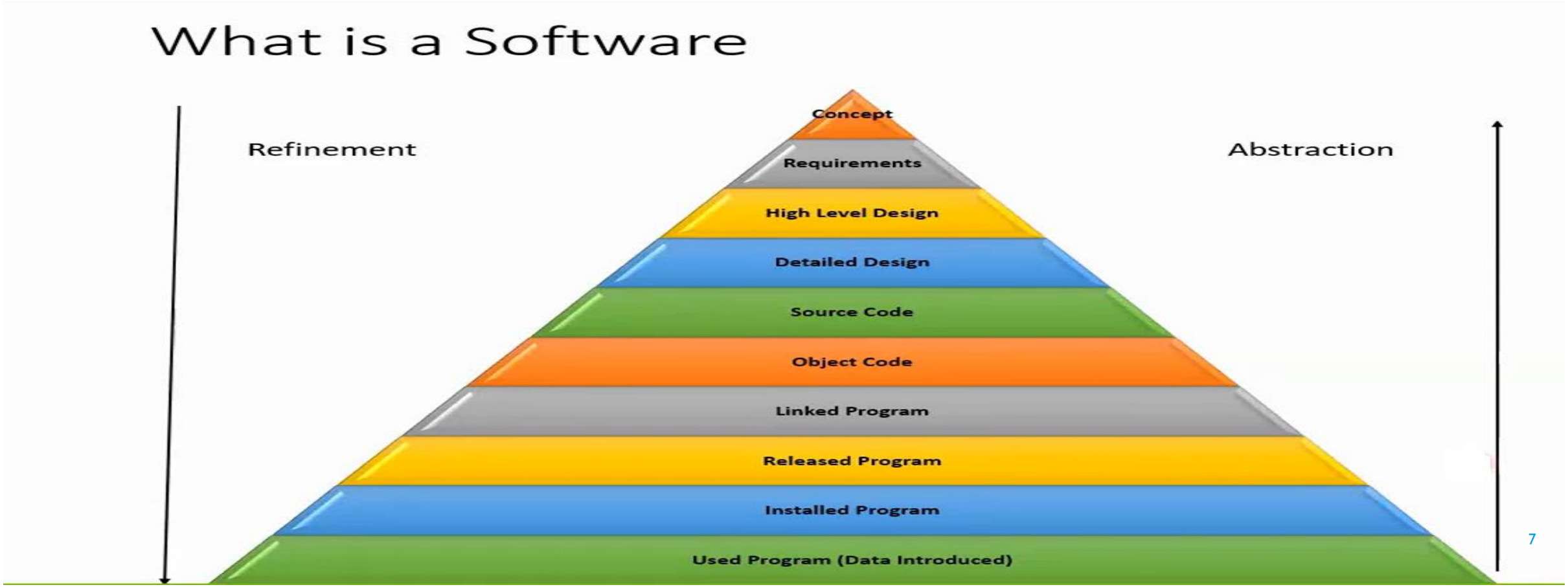
Outline

- General Idea
- Reengineering Concepts
- A General Model for Software Engineering
  - Types of Changes
  - Software Reengineering Strategies
  - Reengineering Variations
- Reengineering Process
  - Reengineering Approaches
  - Source Code Reengineering Reference Model
  - Phase Reengineering Model

Reengineering Concepts

- **Abstraction** and **Refinement** are key concepts used in software development, and both the concepts are equally useful in reengineering.
- It may be recalled that abstraction enables software maintenance personnel to reduce the complexity of understanding a system by:
  I. focusing on the more significant information about the system; and
  II. Hiding the irrelevant details at the moment.
- On the other hand, refinement is the reverse of abstraction.

Reengineering Concepts(Cont.)



Reengineering Concepts (Cont.)

- **Principle of abstraction:** The level of abstraction of the representation of a system can be gradually increased by successively replacing the details with abstract information. By means of abstraction one can produce a view that focuses on selected system characteristics by hiding information about other characteristics.

- **Principle of refinement:** The level of abstraction of the representation of the system is gradually decreased by successively replacing some aspects of the system with more details.

Reengineering Concepts (Cont.)

- A new software is created by going downward from the top, highest level of abstraction to the bottom, lowest level. This downward movement is known as **forward engineering**.

- **Forward engineering** follows a sequence of activities: formulating concepts about the system to identifying requirements to designing the system to implementing the design.

- On the other hand, the upward movement through the layers of abstractions is called **reverse engineering.**

Reengineering Concepts (Cont.)

- **Reverse engineering** of software systems is a process comprising the following steps:
  I. analyze the software to determine its components and the relationships among the components,
  II. represent the system at a higher level of abstraction or in another form.

- **Decompilation** is an example of Reverse Engineering, in which object code is translated into a high-level program.

Reengineering Concepts (Cont.)

- The concepts of abstraction and refinement are used to create models of software development as sequences of phases, where the phases map to specific levels of **abstraction** or **refinement**, as shown in Figure 4.1.
- The four levels are:
  - Conceptual,
  - Requirements,
  - Design, and
  - Implementation.

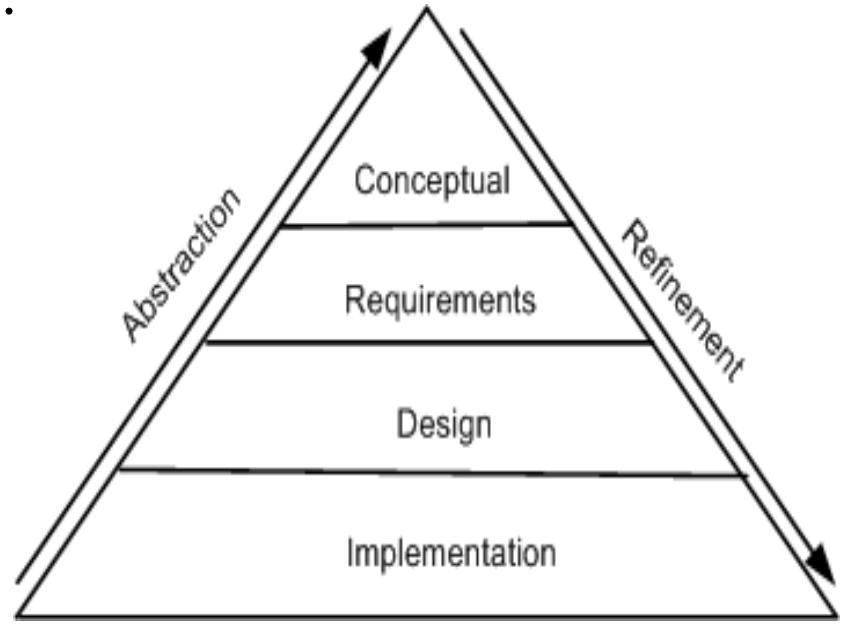


Figure 4.1 levels of abstraction and refinement
© IEEE, 1992

Reengineering Concepts (Cont.)

- The refinement process:

  why? ! what? ! what & how? ! how?

- The abstraction process:

  how? ! what & how? ! what? ! why?

Reengineering Concepts (Cont.)

- An optional principle called **alteration** underlies many reengineering methods.
- **Principle of alteration:** The making of some changes to a system representation is known as alteration. Alteration does not involve any change to the degree of abstraction, and it does not involve modification, deletion, and addition of information.
- **Reengineering principles** are represented by means of arrows. Abstraction is represented by an up-arrow, alteration is represented by a horizontal arrow, and refinement by a down-arrow.
- The arrows depicting refinement and abstraction are slanted, thereby indicating the increase and decrease, respectively, of system information.
- It may be noted that alteration is non-essential for reengineering.

Reengineering Concepts (Cont.)

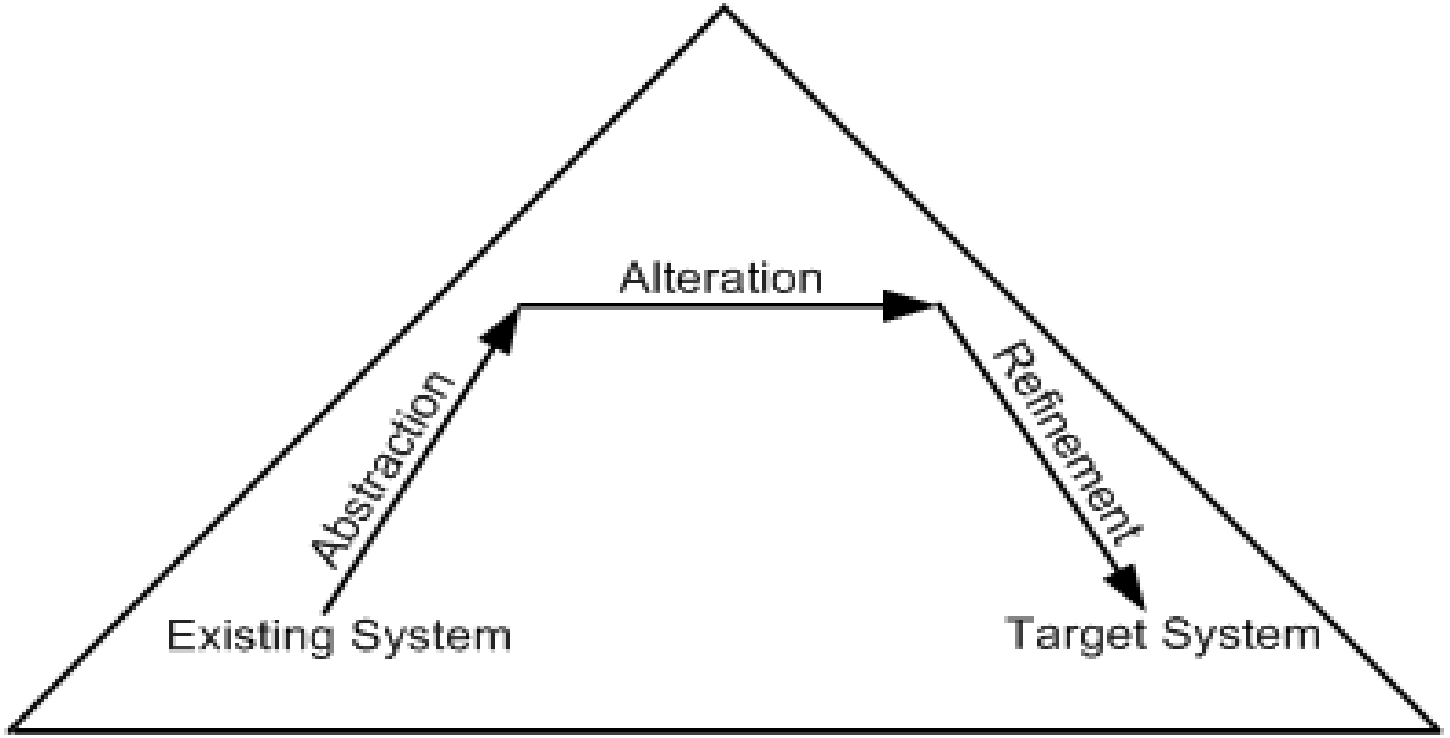


Figure 4.2 Conceptual basis for the reengineering process .

Outline

- General Idea
- Reengineering Concepts
- A General Model for Software Engineering
  - Types of Changes
  - Software Reengineering Strategies
  - Reengineering Variations
- Reengineering Process
  - Reengineering Approaches
  - Source Code Reengineering Reference Model
  - Phase Reengineering Model

A General Model For Software Reengineering

- The reengineering process accepts as input the existing code of a system and produces the code of the renovated system.
- The reengineering process may be as straightforward as translating with a tool the source code from the given language to source code in another language.
- For example, a program written in BASIC can be translated into a new program in C.

A General Model For Software Reengineering

- The reengineering process may be very complex as explained below:
  - recreate a design from the existing source code.
  - find the requirements of the system being reengineered.
  - compare the existing requirements with the new ones.
  - remove those requirements that are not needed in the renovated system.
  - make a new design of the desired system.
  - code the new system.

A General Model For Software Reengineering(Cont.)

- The model in Figure 4.3 proposed by Eric J. Byrne suggests that reengineering is a sequence of three activities:
  - reverse engineering, re-design, and forward engineering
  - strongly founded in three principles, namely, abstraction, alteration, and refinement.

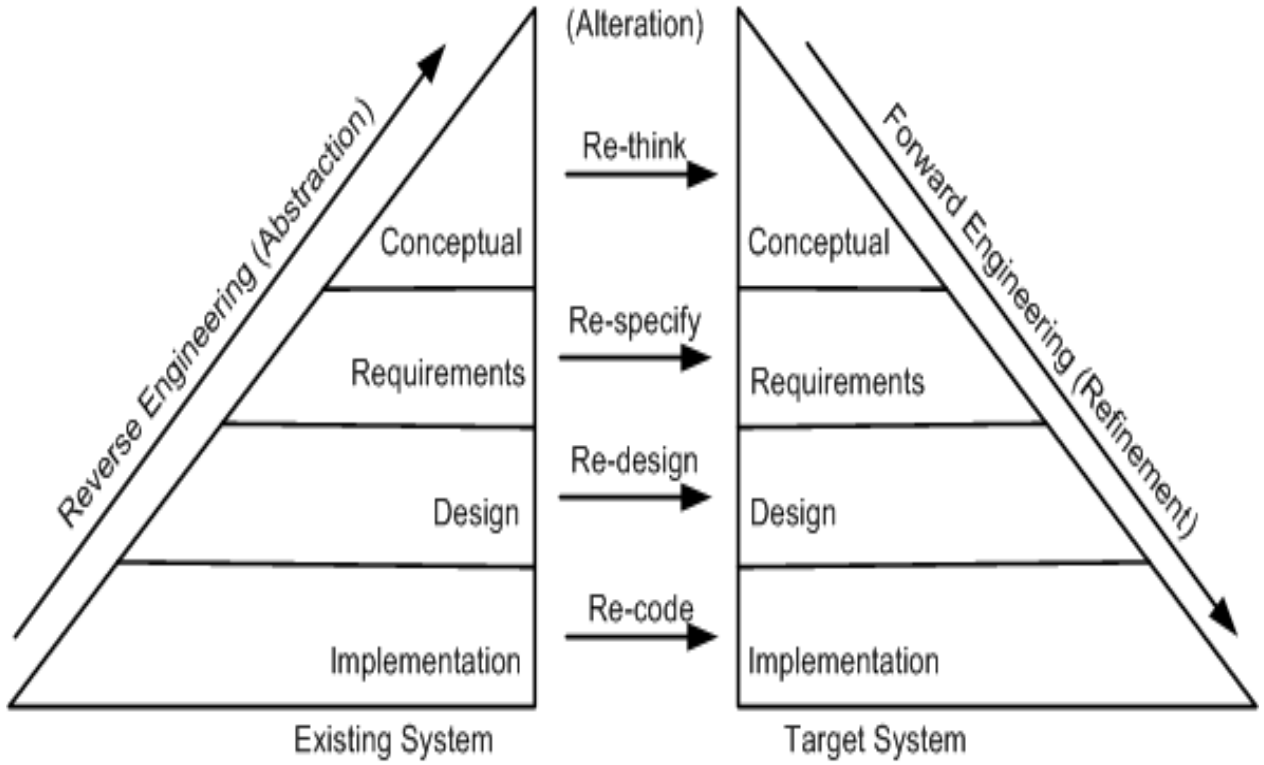


Figure 4.3 General model of software reengineering © IEEE, 1992

A General Model For Software Reengineering(Cont.)

- A visual metaphor called horseshoe, as depicted in Figure 4.4, was developed by Kazman et al. to describe a three-step architectural reengineering process.
- Three distinct segments of the horseshoe are the left side, the top part, and the right side. Those three parts denote the three steps of the reengineering process.

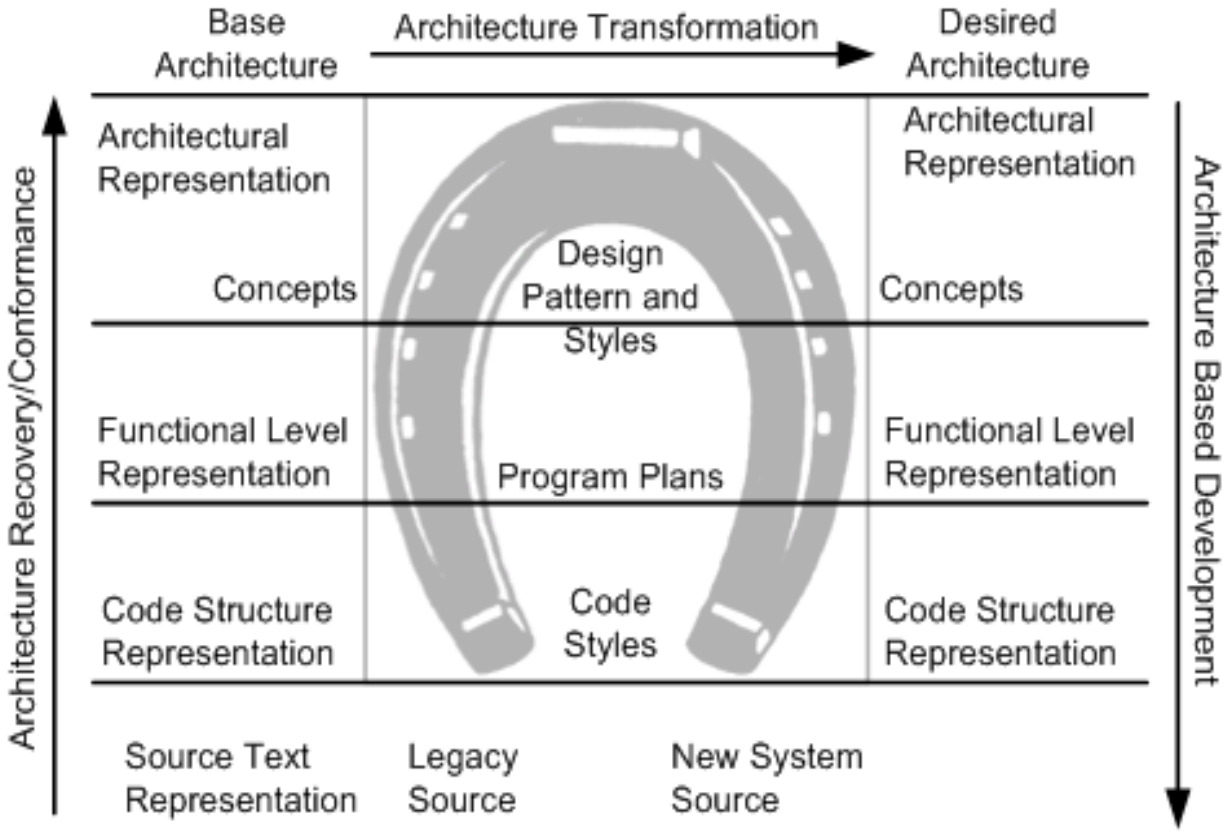


Figure 4.4 Horseshoe model of reengineering

A General Model For Software Reengineering(Cont.)

- Now, we are in a position to re-visit three definitions of reengineering.
  - The definition by Chikofsky and Cross II: Software reengineering is the analysis and alteration of an operational system to represent it in a new form and to obtain a new implementation from the new form. Here, a new form means a representation at a higher level of abstraction.
  - The definition by Byrne: Reengineering of a software system is a process for creating a new software from an existing software so that the new system is better than the original system in some ways.
  - The definition by Arnold: Reengineering of a software system is an activity that: (i) improves the comprehension of the software system, or (ii) raises the quality levels of the software, namely, performance, reusability, and maintainability.

A General Model For Software Reengineering(Cont.)

- In summary, it is evident that reengineering entails:
  I. The creation of a more abstract view of the system by means of some reverse engineering activities,
  II. The restructuring of the abstract view, and
  III. Implementation of the system in a new form by means of forward engineering activities.
- This process is formally captured by Jacobson and Lindstorm with the following expression:

Reengineering = Reverse engineering + Δ + Forward engineering.

A General Model For Software Reengineering(Cont.)

- The element “Δ” captures alterations made to the original system.
- Two major dimensions of alteration are: change in functionality and change in implementation technique.
- A change in functionality comes from a change in the business rules,
- Next, concerning a change of implementation technique, an end-user of a system never knows if the system is implemented in an object-oriented language or a procedural language.

A General Model For Software Reengineering(Cont.)

- Another common term used by practitioners of reengineering is **rehosting**.
- **Rehosting** means reengineering of source code **without addition** or **reduction of features** in the transformed targeted source code.
- **Rehosting** is most effective **when the user is satisfied with the system's functionality**, but looks for better qualities of the system.
- Examples of better qualities are improved efficiency of execution and reduced maintenance costs.

Outline

- General Idea
- Reengineering Concepts
- A General Model for Software Engineering
  - Types of Changes
  - Software Reengineering Strategies
  - Reengineering Variations
- Reengineering Process
  - Reengineering Approaches
  - Source Code Reengineering Reference Model
  - Phase Reengineering Model

Types of Change

Based on the type of changes required, system characteristics are divided into groups: **rethink, respecify, redesign**, and **re-code**.

Recode:

- Implementation characteristics of the source program are changed by re-coding it. Source-code level changes are performed by means of rephrasing and program translation.
- In the latter approach, a program is transformed into a program in a different language. On the other hand, rephrasing keeps the program in the same language
- Examples of translation scenarios are **compilation, decompilation**, and **migration**.
- Examples of rephrasing scenarios are **normalization**, **optimization, refactoring**, and **renovation.**

Types of Change

Redesign:

- The design characteristics of the software are altered by re-designing the system. Common changes to the software design include:
  I. restructuring the architecture;
  II. Modifying the data model of the system; and
  III. replacing a procedure or an algorithm with a more efficient one.

Types of Change (Cont.)

Respecify:

- This means changing the requirement characteristics of the system in two ways:
  I. change the form of the requirements, and
  II. change the scope of the requirements.

Rethink:

- Re-thinking a system means manipulating the concepts embodied in an existing system to create a system that operates in a different problem domain.
- It involves changing the conceptual characteristics of the system, and it can lead to the system being changed in a fundamental way.
- Moving from the development of an ordinary cellular phone to the development of smartphone system is an example of Re-think.

Outline

- General Idea
- Reengineering Concepts
- A General Model for Software Engineering
  - Types of Changes
  - Software Reengineering Strategies
  - Reengineering Variations
- Reengineering Process
  - Reengineering Approaches
  - Source Code Reengineering Reference Model
  - Phase Reengineering Model

Software Reengineering Strategies

Three strategies that specify the basic steps of reengineering are **rewrite**, **rework**, and **replace.**

Rewrite strategy:

This strategy reflects the principle of alteration. By means of alteration, an operational system is transformed into a new system, while preserving the abstraction level of the original system. For example, the Fortran code of a system can be rewritten in the C language.

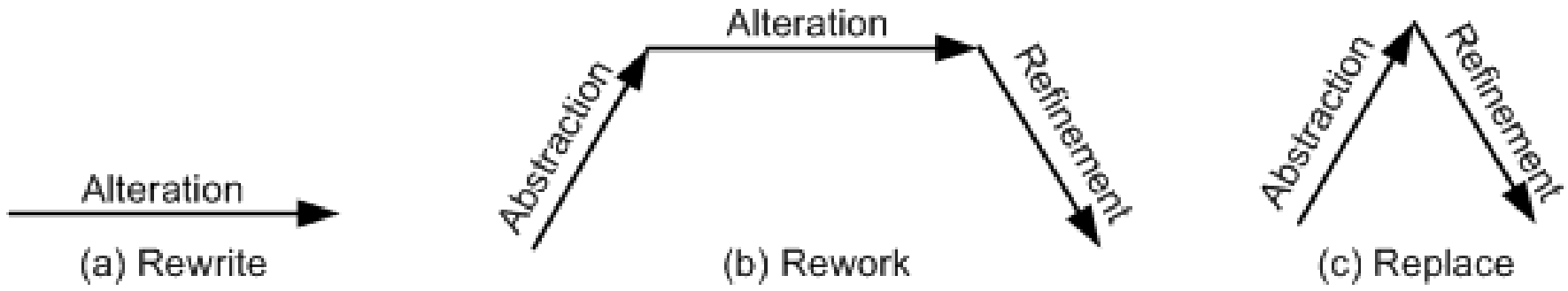


Figure 4.5 Conceptual basis for reengineering strategies

Software Reengineering Strategies

Rework strategy:

- The rework strategy applies all the three principles.
- Let the goal of a reengineering project is to replace the unstructured control flow constructs, namely GOTOs, with more commonly used structured constructs, say, a “for” loop.
- A classical, rework strategy based approach is as follows:
  - Application of abstraction: By parsing the code, generate a control-flow graph (CFG) for the given system.
  - Application of alteration: Apply a restructuring algorithm to the control-flow graph to produce a structured control-flow graph.
  - Application of refinement: Translate the new, structured control-flow graph back into the original programming language.

Software Reengineering Strategies

Replace strategy:

- The replace strategy applies two principles, namely, abstraction and refinement.
- To change a certain characteristic of a system:
  I. the system is reconstructed at a higher level of abstraction by hiding the details of the characteristic; and
  II. a suitable representation for the target system is generated at a lower level of abstraction by applying refinement.
- Let us reconsider the GOTO example. By means of abstraction, a program is represented at a higher level without using control flow concepts.
- Next, by means of refinement, the system is represented at a lower level of abstraction with a new structured control flow.

Thank you